CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[Section]
System Integration and Architecture

System Design Document (SDD)

Project Title: SnackDash

Prepared By: Relacion, Reden Misael Palamos

Version: 2.0

Date: 3/2/2026

Status: Draft

EXECUTIVE SUMMARY

1.1 Project Overview

SnackDash is a multi-tenant campus food ordering system designed to reduce waiting time in university food stalls. The system enables students to browse menus from multiple food stalls, place digital orders, and track real-time order status.

SnackDash uses:

- Node.js with Express (Backend API)
- MySQL Database (via XAMPP)
- RESTful API communication
- JWT authentication
- Multi-tenant architecture

The system allows independent stall owners to manage their own menus and orders securely within a shared database environment.

1.2 Objectives

1. Digitize campus food ordering.
2. Reduce congestion during peak hours.
3. Provide stall owners with a real-time order dashboard.
4. Implement secure multi-tenant data separation.
5. Develop a scalable backend using Node.js and MySQL.

1.3 Scope

Included Features

- User registration and login
- Role-based access control (Student, Owner, Admin)
- Stall management
- Menu management
- Order placement
- Wallet payment simulation

- Order status tracking
- Multi-tenant filtering

Excluded Features

- External payment gateways
- Food delivery system
- Push notifications
- SMS integration

1.0 INTRODUCTION

1.1 Purpose

This document defines the technical design and architecture of SnackDash. It serves as the official reference for development, testing, and deployment.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: SnackDash
Domain: Campus Food Ordering System
Primary Users:

- Students
- Stall Owners
- Admin

Problem Statement:
Students experience long queues during break hours.

Solution:
A centralized digital ordering system that allows advance ordering and real-time status tracking.

2.2 Core User Journeys

Student Order Flow

1. Register/Login
2. Browse available stalls
3. Select a stall
4. View menu items
5. Add items to cart
6. Confirm order
7. Wallet balance deducted
8. Track order status

Order Status Flow:

- PENDING
- COOKING
- READY
- COMPLETED

Stall Owner Flow

1. Login
2. View incoming orders
3. Update order status
4. Manage menu items
5. Open/Close stall

Admin Flow

1. Login
2. View users
3. View stalls

4. Monitor overall system activity

2.3 Feature List (MoSCoW)

MUST HAVE

- Authentication system
- Role-based middleware
- Stall CRUD
- Menu CRUD
- Order placement
- Wallet deduction
- Order status updates

SHOULD HAVE

- Order history
- Stall availability toggle

WON'T HAVE

- External payments
- Delivery tracking

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance

- API response ≤ 2 seconds
- Support at least 300 concurrent users
- Indexed MySQL tables for faster queries

3.2 Security

- Password hashing using bcrypt
- JWT-based authentication

- Role-based authorization middleware
- SQL injection prevention using parameterized queries

3.3 Reliability

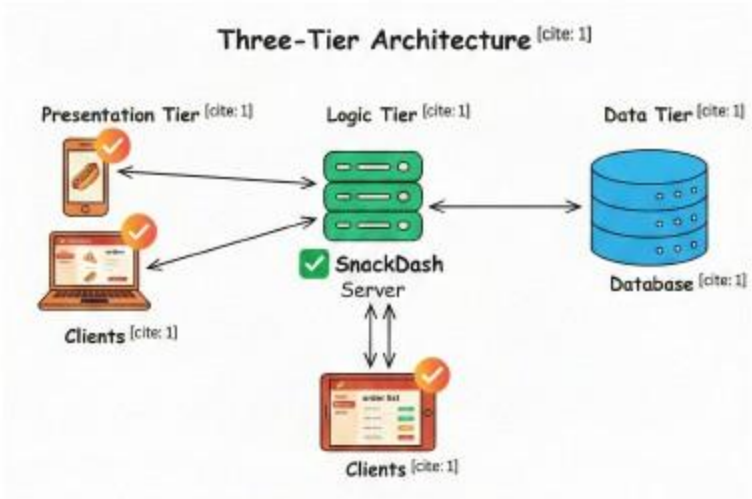
- MySQL foreign key constraints
- Centralized error handling in Express
- Transaction handling during order creation

3.4 Scalability

- Shared database with tenant filtering
- Modular folder structure
- RESTful API structure

4.0 SYSTEM ARCHITECTURE

4.1 Architecture Model



SnackDash Application Technology Stack:

- **Technology Stack:**
- **Development Environment:**
- Visual Studio Code (VS Code) [cite: 1]
- **Backend (adapted):**
- Java 17 (or Java 21 LTS), Spring Boot 3.x, Spring Security, Spring Data JPA [cite: 1]
- **Database:**
- PostgreSQL 14+ [cite: 1]
- **Web Frontend:**
- React 18, TypeScript, Tailwind CSS, Axios [cite: 1]
- **Mobile (Student & Owner):**
- Kotlin, Jetpack Compose, Retrofit [cite: 1]
- **Build Tools:**
- Maven (Backend), npm (Web), Gradle (Android) [cite: 1]
- **Deployment:**
- Localhost (Development) [cite: 1]
- Railway / Render (Production – optional) [cite: 1]
- Vercel (Web – optional) [cite: 1]
- APK (Mobile) [cite: 1]

10

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL:

http://localhost:5000/api/v1

Authentication:

Authorization: Bearer <JWT_TOKEN>

Data Format:

- JSON

5.2 Authentication Endpoints

POST /auth/register
POST /auth/login

5.3 Stall Endpoints

GET /stalls
POST /stalls
PUT /stalls/:id
DELETE /stalls/:id

5.4 Menu Endpoints

GET /stalls/:stallId/menu
POST /menu
PUT /menu/:id
DELETE /menu/:id

5.5 Order Endpoints

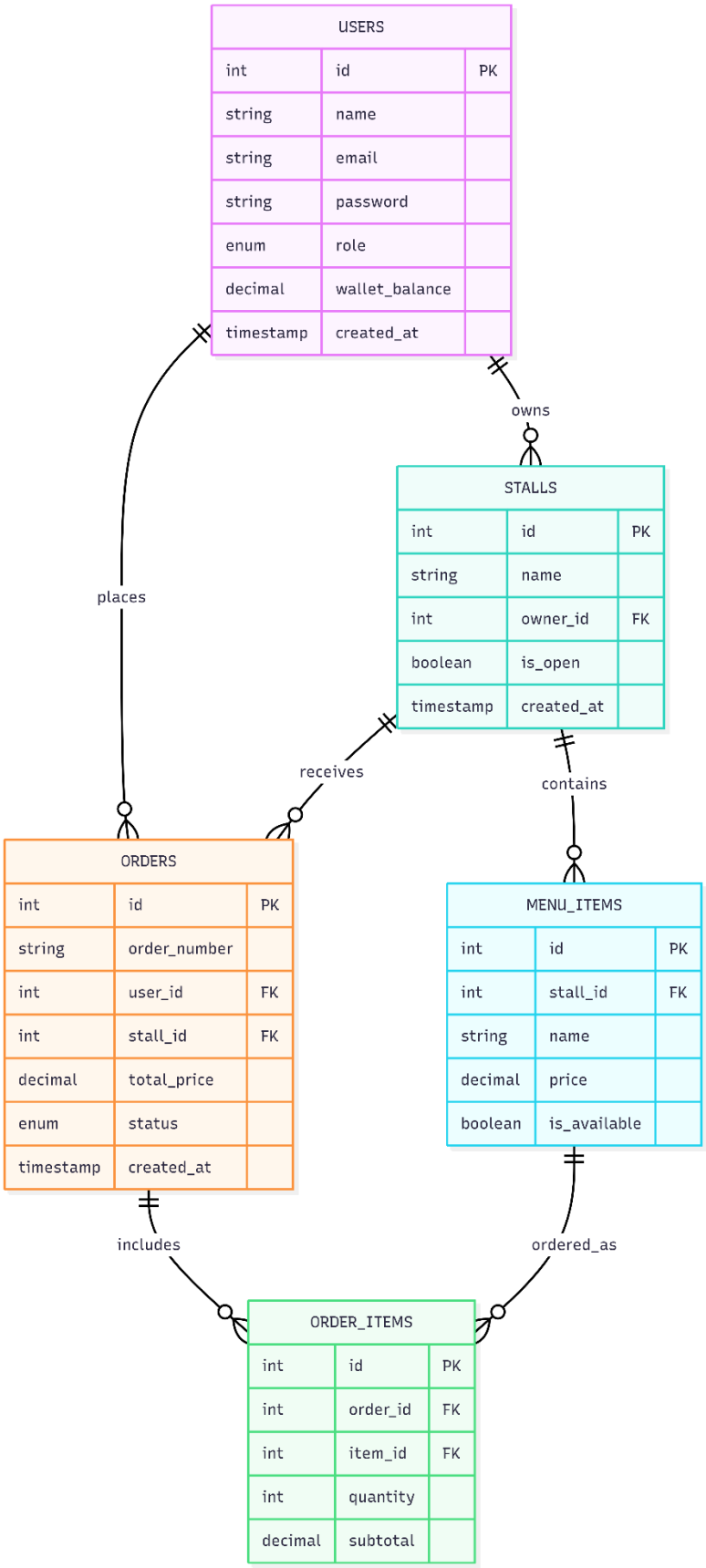
POST /orders
GET /orders/:id
GET /my-orders
PUT /orders/:id/status

5.6 Standard API Response Format

```
{
  "success": true,
  "data": {},
  "error": null,
  "timestamp": "2026-03-02T10:30:00Z"
}
```

6.0 DATABASE DESIGN (MySQL – XAMPP)

6.1 Tables

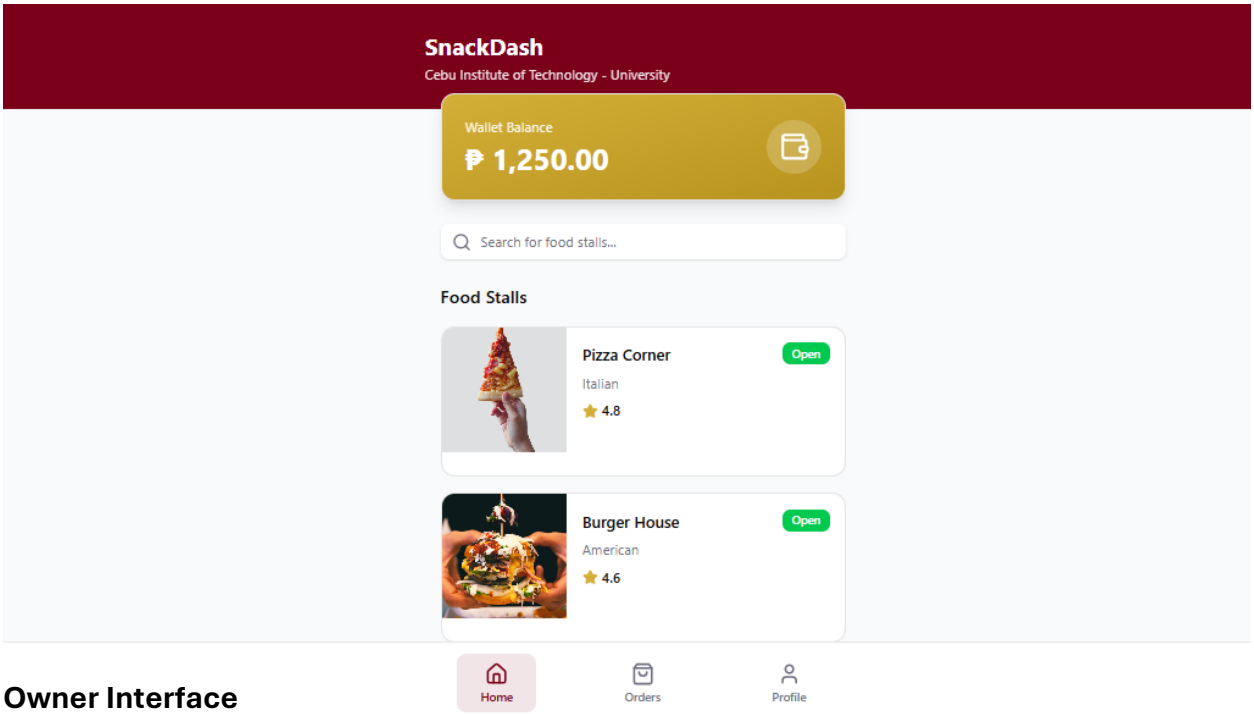

USERS
int id PK
string name
string email
string password
enum role
decimal wallet_balance
timestamp created_at
owns
STALLS
int id PK
string name
int owner_id FK
boolean is_open
timestamp created_at
places
receives
contains
ORDERS
int id PK
string order_number
int user_id FK
int stall_id FK
decimal total_price
enum status
timestamp created_at
MENU_ITEMS
int id PK
int stall_id FK
string name
decimal price
boolean is_available
includes
ordered_as
ORDER_ITEMS
int id PK
int order_id FK
int item_id FK
int quantity
decimal subtotal

6.2 Relationships

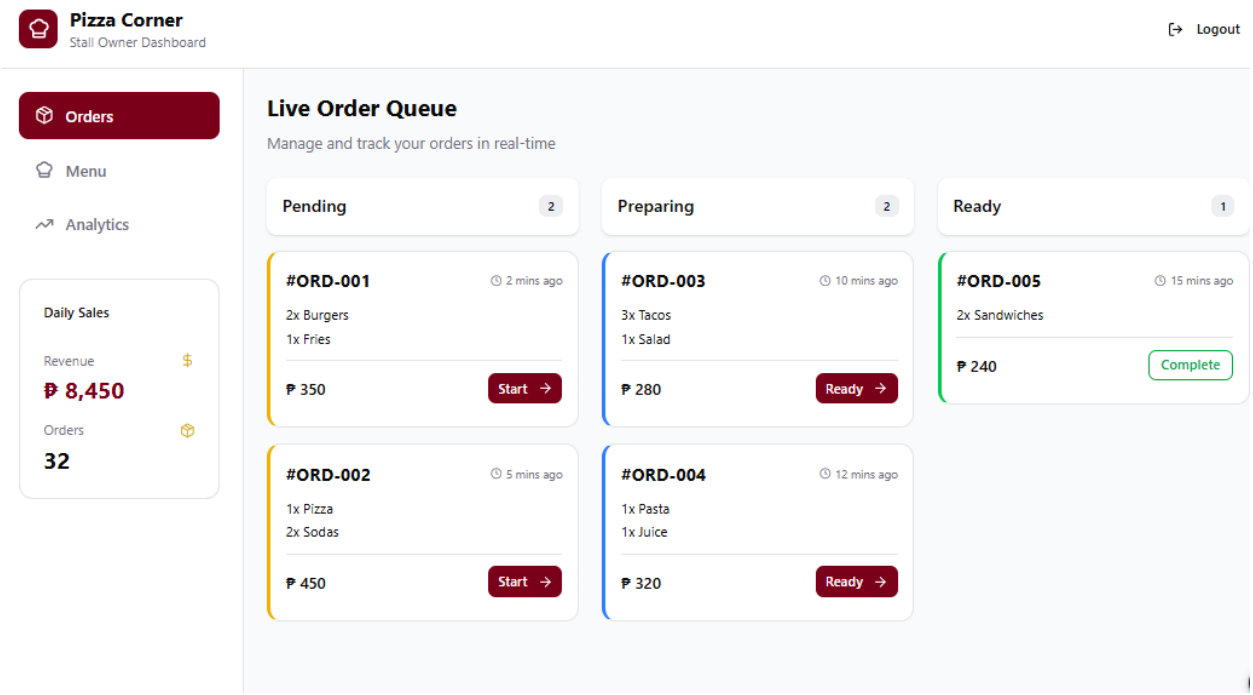
- One User → Many Orders
- One Owner → Many Stalls
- One Stall → Many Menu Items
- One Stall → Many Orders
- One Order → Many Order Items

7.0 UI/UX DESIGN

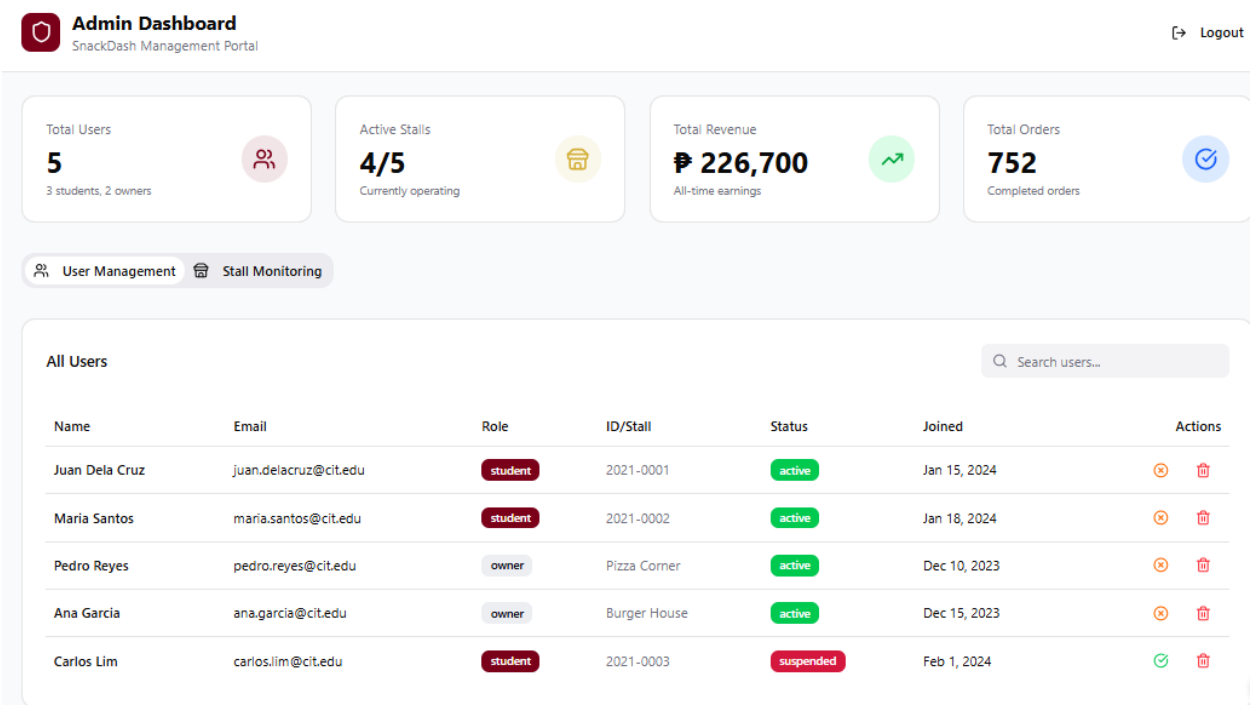
Student Interface



Owner Interface



Admin Interface



Design Principles:

- Responsive layout
- Clean UI

- Clear status color indicators
- Mobile-friendly

8.0 PROJECT PLAN

Phase 1 – Setup

- Install XAMPP
- Create MySQL database
- Initialize Node.js project
- Configure Express

Phase 2 – Authentication

- Register/Login
- JWT middleware
- Role middleware

Phase 3 – Stall & Menu

- Stall CRUD
- Menu CRUD

Phase 4 – Orders

- Cart logic
- Order creation
- Wallet deduction
- Status update

Phase 5 – Testing & Deployment

- API testing
- Bug fixing
- Final integration